AOSP 进阶丛书 · 第十八册 · ANDROID 13 (API 33)

CTS 认证实战指南

CTS 结构（200-300 万断言）、Tradefed 调度引擎、VTS HAL 验证、厂商适配实战
——Google Play 预装的唯一通行证。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

目 录

前言：兼容性的唯一权威 2
第一章 CTS 的结构与覆盖面 2
第二章 Tradefed：测试调度引擎 2
第三章 VTS：HAL 的验证 3
第四章 开发者视角：怎么适配 3
第五章 设计决策五问 4
附录 A 实战命令 4
附录 B 快问快答（10 条） 4
附录 C 自检题 5
教科书习题集（分级） 5
术语表 5
排障手册：CTS 认证问题定位 5

本册速览

CTS 结构：cts/tests/tests/ 下数千测试用例
Tradefed：测试调度框架（device selection/test execution/result aggregation）
VTS：HAL 验证（书14 的 AIDL 接口逐个过）
测试流程：下载 CTS→跑→提交 Google 审→认证
设计动机：为什么 CTS 是兼容性的唯一权威

前言：兼容性的唯一权威

CTS（Compatibility Test Suite）是 Google 认证一台 Android 设备的唯一标准——**通过 CTS = 拿到 GMS（Google Play）预装权**。本册讲 CTS 的结构（测什么）、Tradefed 怎么跑、VTS（HAL 验证）与 CTS 的关系、以及开发者视角的"怎么适配"。

源码基线不变。主角：`cts/`（测试集）、`cts/tools/tradefed/`（调度框架）。

第一章 CTS 的结构与覆盖面

text cts/tests/tests/（本树实际布局）

```
├── app/（Activity/Service/Notification…数千用例）
├── app.usage/ • accounts/ • animation/ • database/ • location/…
├── net/ • os/ • permission/ • security/ ├── telephony/ • view/ • widget/
└── … 按模块分组
每个用例 = junit Test Class + AndroidManifest（声明 targetSdk）
// 讲解注释：CTS 的核心断言是"framework API 的行为符合 SDK 文档"——
// 任何 deviation 都是 fail。
```

1.1 一个测试用例的样子

java cts/tests/tests/app/src/android/app/cts（节选+讲解注释）

```
// 典型 CTS 用例的结构（以 ActivityTest 为例）
@SmallTest
public class ActivityTest extends ActivityInstrumentationTestCase2<MyActivity> {
public void testActivityLifecycle() {
// 讲解注释①：启动 Activity→断言 onCreate→onStart→onResume 顺序
// 讲解注释②：断言基于 android.test 框架（junit3 风格为主）
// 讲解注释③：通过 = 行为与 SDK 文档一致
}
}
// 讲解注释④：一个 CTS 包约 200-300 万个断言，
// 跑完一台设备 8-72 小时（取决于模块）。
```

第二章 Tradefed：测试调度引擎

text cts/tools/tradefed/host/src/…（结构+讲解注释）

```
TestInvocation（入口：解析命令→选设备→跑→聚合结果）
├── DeviceMonitor（设备状态：USB/温度/存储）
├── TestRunner（执行器：逐用例跑→捕获失败→截图/日志）
├── ResultReporter（报告：XML/HTML 每用例结果）
└── RetryFilter（重试：已知 flaky 的自动重跑）
// 讲解注释：Tradefed 是独立可执行 jar（cts-tradefed 命令）——
// 不依赖 AOSP 构建，预构建发行（cts.zip 内）。
```

2.1 实战流程

```bash
# 下载 CTS（Google 官方发布，非源码编译）
# 解压→运行：
cts-tradefed run cts --plan CTS -s <serial> # 跑完整 CTS
cts-tradefed run cts -m CtsAppTest -s <serial> # 只跑 app 模块
# 结果：testResult.xml + screenshots/ + logcat/ → 提交 Google 审
```

第三章 VTS：HAL 的验证

VTS（Vendor Test Suite）验证 HAL 接口实现正确性——**书14 写的 HAL 必须过 VTS**：

```text VTS 与 CTS 的分工
CTS：framework API 行为（App 视角）
VTS：HAL 接口（书14 AIDL HAL 的每个方法调用是否按规范返回）
└── 本树 system/vts/ + test/vts/
GTS：Google 专有（GMS 验证——闭源发布）
```

第四章 开发者视角：怎么适配

场景	要过什么	常见 fail 原因
厂商 HAL	VTS	返回值不符合 .aidl 定义/超时
定制 framework	CTS	行为偏离 SDK 文档
改了 SELinux 策略	CTS+VTS	avc denied 导致服务起不来
GMS 预装	CTS+GTS	所有通过后才拿 Play Store 权

先修要求与学习路线

先修：已读第一册（架构总览）。

读完本书你将能够：

- 解释CTS 认证的核心机制与设计决策
- 追踪关键调用链并标注每次跨进程跳
- 使用 dumpsys/日志定位本领域问题
- 回答书7 对应章节的面试题

> **设计洞察 · 面试视角："CTS 过不了怎么办"**
> 定位：testResult.xml 找 fail 用例→看截图/日志→分析是 framework 偏离还是测试环境 flaky→修代码或申请 waiver（Google 审批豁免）。

第五章 设计决策五问

- **为什么 CTS 是唯一权威而非文档？** 文档可被误读；代码断言=机器可验证。CTS 通过=所有 App 在你设备上行为正确（Play Store 的信任前提）。
- **为什么 Tradefed 独立于 AOSP 构建？** 发行节奏不同（CTS 每月新版与 Android 版本绑定）；测试框架闭源演进。
- **为什么 VTS 与 CTS 分开？** 覆盖层不同（HAL vs framework）；厂商定制主要影响 HAL——分开发行更精准。
- **为什么 CTS 那么久？** 200-300 万断言+逐用例等待 UI 渲染+网络超时容忍——可在多台设备分片并行（shard）。
- **为什么 flaky 用例需要 RetryFilter？** UI 测试受设备状态影响（温度/存储/网络波动）——已知 flaky 重跑一次降低误报。

附录 A 实战命令

目标	命令	耗时
完整 CTS	cts-tradefed run cts --plan CTS	8-72h
单模块	cts-tradefed run cts -m CtsAppTest	分钟级
单测试	cts-tradefed run cts -t android.app.cts.ActivityTest#testLifecycle	秒级
重跑失败	cts-tradefed run cts --retry <session_id>	智能
VTS	vts-tradefed run vts -m VtsHalPowerV1_0Target	分钟级
结果	查看 testResult.xml / results/<session>/	—

附录 B 快问快答（10 条）

- CTS 全称？——Compatibility Test Suite。
- CTS 通过意味着什么？——可以拿 GMS 预装权（Play Store）。
- CTS 大约多少断言？——200-300 万。
- Tradefed 是什么？——测试调度框架（选设备→跑→聚合）。
- VTS 测什么？——HAL 接口（CTS 测 framework API）。

- GTS 是什么？——Google 专有（GMS 验证）。
- CTS 跑多久？——8-72 小时（可分片并行）。
- fail 了怎么办？——分析 testResult.xml→修代码或申请 waiver。
- 谁跑 CTS？——设备厂商（在 Google 实验室或自建环境）。
- App 开发者需要过 CTS 吗？——不需要（CTS 测 framework 行为）。

附录 C 自检题

- CTS/VTS/GTS 三者的覆盖面与发行方。
- Tradefed 的执行流水线四段。
- 厂商"改了 HAL 忘了过 VTS"的后果链。

教科书习题集（分级）

第一级：简答题（检验概念理解）

（从附录 B 快问快答中挑 5 题，闭卷手写答案。）

第二级：分析题（检验机制理解）

- **追踪调用链**：画出本册核心机制的完整调用图。
- **失败推演**：核心机制某步失败，系统会怎样？
- **跨册串联**：本册机制与 Binder 十跳（书3）的交汇点。

第三级：设计题

- **架构演进**：重构本册核心机制，你会怎么改？
- **排障方案**：设计分层定位方案。

术语表

（术语表：从附录 B 提取本册专属术语，按拼音排序。）

排障手册：CTS 认证问题定位

1. dumpsys 定位状态 → 2. logcat 查看日志 → 3. 深入各章。

深度实战：CTS 认证的排障场景分析

场景：CTS 用例莫名 fail

1. 看结果 XML：result="fail" 的原因码
2. 看 logcat 中该用例的输出（ Tradefed 会保留）
3. 检查设备状态（温度/存储/网络）→ 可能是 flaky
4. 用 --retry 重跑
5. 如果稳定 fail → 分析 framework 行为 vs SDK 文档的偏离